

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int info;
    struct node *next;
};

// Insert at beginning
struct node* insertslbeg(struct node *start, int item) {
    struct node *p;
    p = (struct node *)malloc(sizeof(struct node));
    p->info = item;
    p->next = start;
    start = p;
    return start;
}

// Insert at end
struct node* insertslend(struct node *start, int item) {
    struct node *p, *temp;
    p = (struct node *)malloc(sizeof(struct node));
    p->info = item;
    p->next = NULL;
    if (start == NULL) {
        start = p;
    } else {
        temp = start;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = p;
    }
    return start;
}

```

// Insert at any position

```
struct node* insertllanypos(struct node *start, int item, int r) {
```

```
    struct node *p, *temp;
```

```
    p = (struct node *)malloc(sizeof(struct node));
```

```
    p->info = item;
```

```
    if (r == 1) {
```

```
        p->next = start;
```

```
        start = p;
```

```
    } else {
```

```
        temp = start;
```

```
        for (int i = 1; i < r - 1 && temp != NULL; i++) {
```

```
            temp = temp->next;
```

```
        }
```

```
        if (temp != NULL) {
```

```
            p->next = temp->next;
```

```
            temp->next = p;
```

```
        } else {
```

```
            printf("Invalid position\n");
```

```
        }
```

```
    return start;
```

```
}
```

// Display the list

```
void display(struct node *start) {
```

```
    struct node *temp = start;
```

```
    printf("Linked List: ");
```

```
    while (temp != NULL) {
```

```
        printf("%d -> ", temp->info);
```

```
        temp = temp->next;
```

```
    }
```

```
    printf("NULL\n");
```

```
}
```

```

// Main function

int main() {
    struct node *start = NULL;

    int choice, item, r, n;

    printf("Enter number of nodes to insert: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\nChoose insertion method:\n");
        printf("1. Insert at beginning\n");
        printf("2. Insert at end\n");
        printf("3. Insert at any position\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        printf("Enter value to insert: ");
        scanf("%d", &item);
        if (choice == 1) {
            start = insertllbeg(start, item);
        } else if (choice == 2) {
            start = insertllend(start, item);
        } else if (choice == 3) {
            printf("Enter position to insert: ");
            scanf("%d", &r);
            start = insertllanypos(start, item, r);
        } else {
            printf("Invalid choice\n");
        }
    }

    printf("\nFinal linked list:\n");
    display(start);
    return 0;
}

```

OUTPUT

```
Enter number of nodes to insert: 6

Choose insertion method:
1. Insert at beginning
2. Insert at end
3. Insert at any position
Enter your choice: 1
Enter value to insert: 10

Choose insertion method:
1. Insert at beginning
2. Insert at end
3. Insert at any position
Enter your choice: 1
Enter value to insert: 20

Choose insertion method:
1. Insert at beginning
2. Insert at end
3. Insert at any position
Enter your choice: 2
Enter value to insert: 30

Choose insertion method:
1. Insert at beginning
2. Insert at end
3. Insert at any position
Enter your choice: 3
Enter value to insert: 50
Enter position to insert: 4

Choose insertion method:
1. Insert at beginning
2. Insert at end
3. Insert at any position
Enter your choice: 1
Enter value to insert: 70

Choose insertion method:
1. Insert at beginning
2. Insert at end
3. Insert at any position
Enter your choice: 2
Enter value to insert: 90

Final linked list:
Linked List: 70 -> 20 -> 10 -> 30 -> 50 -> 90 -> NULL

Process returned 0 (0x0)   execution time : 41.833 s
Press any key to continue.
```